

前言

实验流程

环境交代

代码下载

修改配置

修改数据库配置

1、建库

2、建表

3、修改yml中数据库连接配置

修改缓存配置

修改日志配置

修改日志路径

修改字符集

本地运行

项目打包

前端项目打包

1、安装依赖

2、构建打包

后端项目打包

打jar包

打war包

前端项目部署

后端项目部署

jar包部署方式

war包部署方式

配置Nginx代理和转发

后台项目多实例部署

后记

前言

关于「**如何将一个前后端分离的开源项目部署到自己的服务器上**」这个话题很早之前我就想做了。

一来是因为有很多初学的小伙伴们反馈说需要；而且经常也听到一部分小伙伴反馈说自己的云服务器买了不知道干啥，有什么办法可以玩起来，等等之类的话题。

所以今天这个坑必须给填上了，我这里又写了一个「**服务器项目部署PDF示例文档**」，希望对小伙伴们有用。

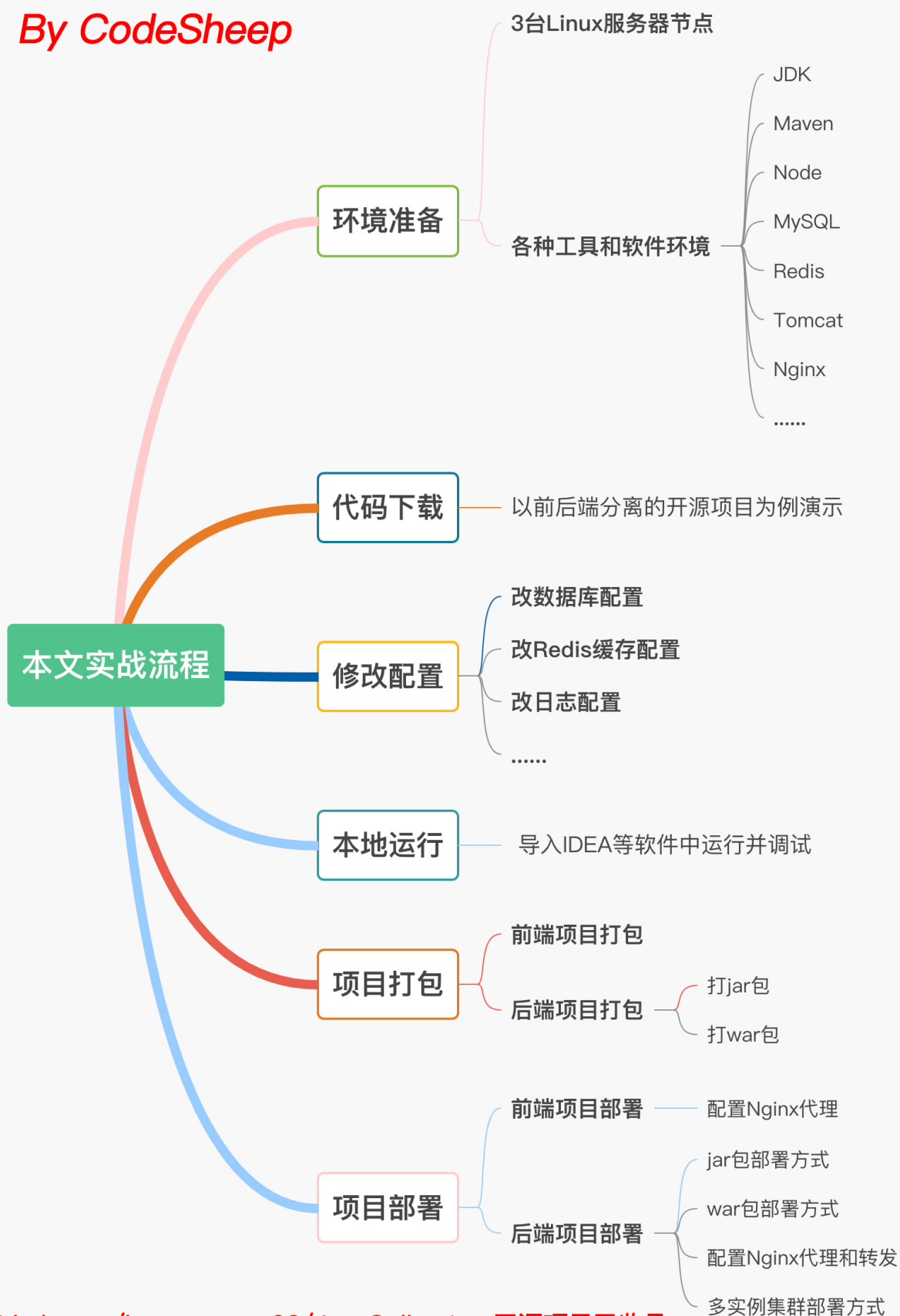
注：本文档在 Github开源项目：github.com/hansonwang99/JavaCollection 中已收录，有详细自学编程学习路线、面试题和面经、编程资料及系列技术文章等，资源持续更新中...

另外，联系作者、提意见，可以参看本文档尾部，有相应联系方式。

实验流程

我还是先把本文的实验流程写在前面，并且画了个思维导图，这样好知道后面每一步在干嘛。

By CodeSheep



github.com/hansonwang99/JavaCollection 开源项目已收录

然后我们接下来一项项安排即可。

环境交代

部署离不开服务器节点、Linux系统和一系列软件环境以及基础设施，这个在之前就已经出过文章、文档以及视频聊过了，具体可以参看该系列文章：

- 服务器软件大科普
- 人手一套Linux环境搭建之：macOS版本教程
- 人手一套Linux环境搭建之：Windows版本教程
- 熬10天夜，肝出了这个PDF版「软件安装手册」

这些是本文实战的前提。

本实验投入了三台Linux主机节点，安排如下：

名称	IP地址	包含软件	用途
节点1	192.168.31.100	Nginx、Node、Git、MySQL、Redis	部署前端项目
节点2	192.168.31.101	Git、JDK、Maven、Tomcat	部署后端项目实例1
节点3	192.168.31.102	Git、JDK、Maven、Tomcat	部署后端项目实例2

顺便详细交代一下本文所用工具和软件的版本情况：

名称	版本	用途
Linux	CentOS 7.4 64bit	服务器操作系统
Git	1.8.3.1	代码下载
JDK	1.8.0_161	Java运行环境
Node	v12.16.3	
npm	6.14.4	前端项目构建和打包
Maven	3.6.3	后端项目构建和打包
MySQL	5.7.30	数据库
Redis	5.0.8	缓存
Nginx	1.17.10	Web服务器
Tomcat	8.5.55	应用服务器

代码下载

本文实验采用优秀开源项目Ruoyi的前后端分离版作为实验载体，我们将它完整的部署起来。

项目地址：https://gitee.com/y_project/RuoYi-Vue。

直接用 `git` 命令下载到本地某个目录下即可。

```
git clone https://gitee.com/y_project/RuoYi-Vue
```

下载完成的代码目录里包含两个子目录，分别为**前端项目**目录和**后端项目**目录。

我这里在本地宿主机和三个 `Linux` 节点上都各下载了一份项目源码，路径分别为：

- 本地代码：`/Users/codesheep/IdeaProjects/RuoYi-Vue`
- 节点1代码路径：`/root/workspace/RuoYi-Vue`
- 节点2代码路径：`/root/workspace/RuoYi-Vue`
- 节点3代码路径：`/root/workspace/RuoYi-Vue`

本地代码是为了稍后导入 `IDEA` 中方便运行、调试和代码阅读及修改，`Linux` 系统节点上的三份代码待会直接打包部署使用。

修改配置

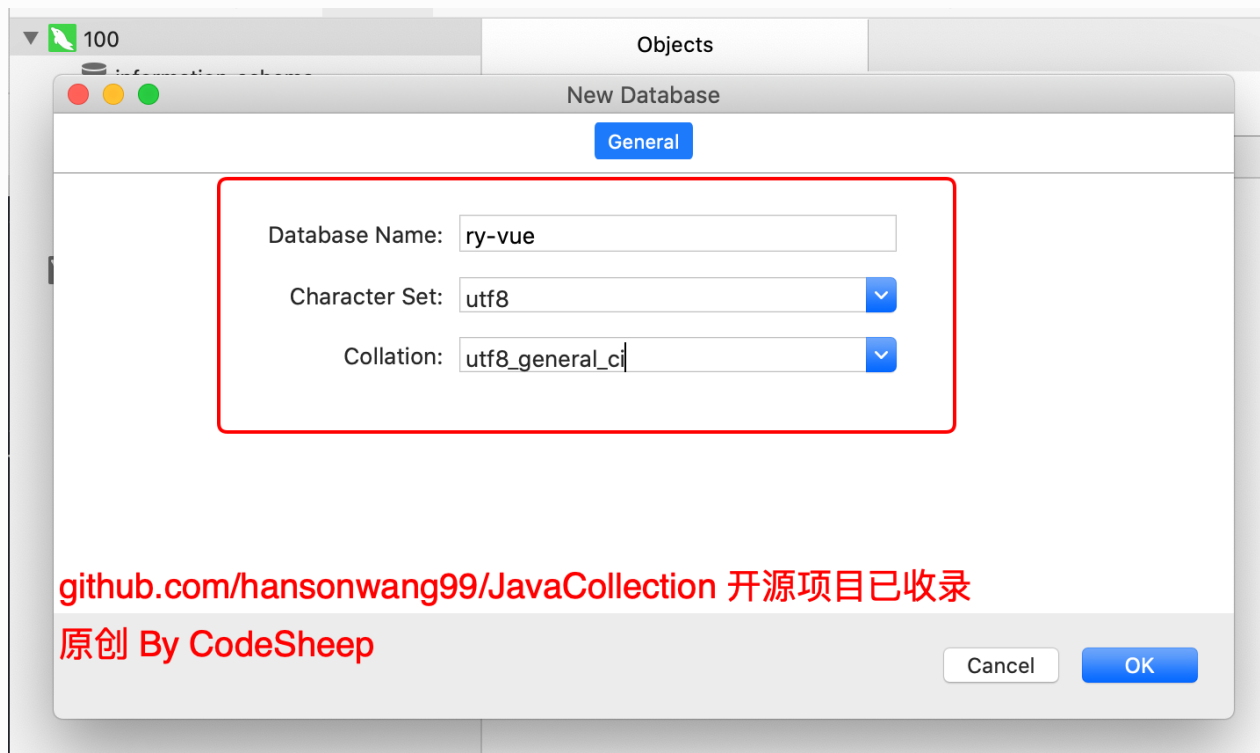
一个开源项目从网上下载之后，想直接立马就能完美运行起来不太现实，毕竟还有很多东西要根据自己的实际情况进行配置，比如数据库、缓存、日志路径等等，都需要根据自己实际情况修改。

所以我们接下来一一配置。

修改数据库配置

1、建库

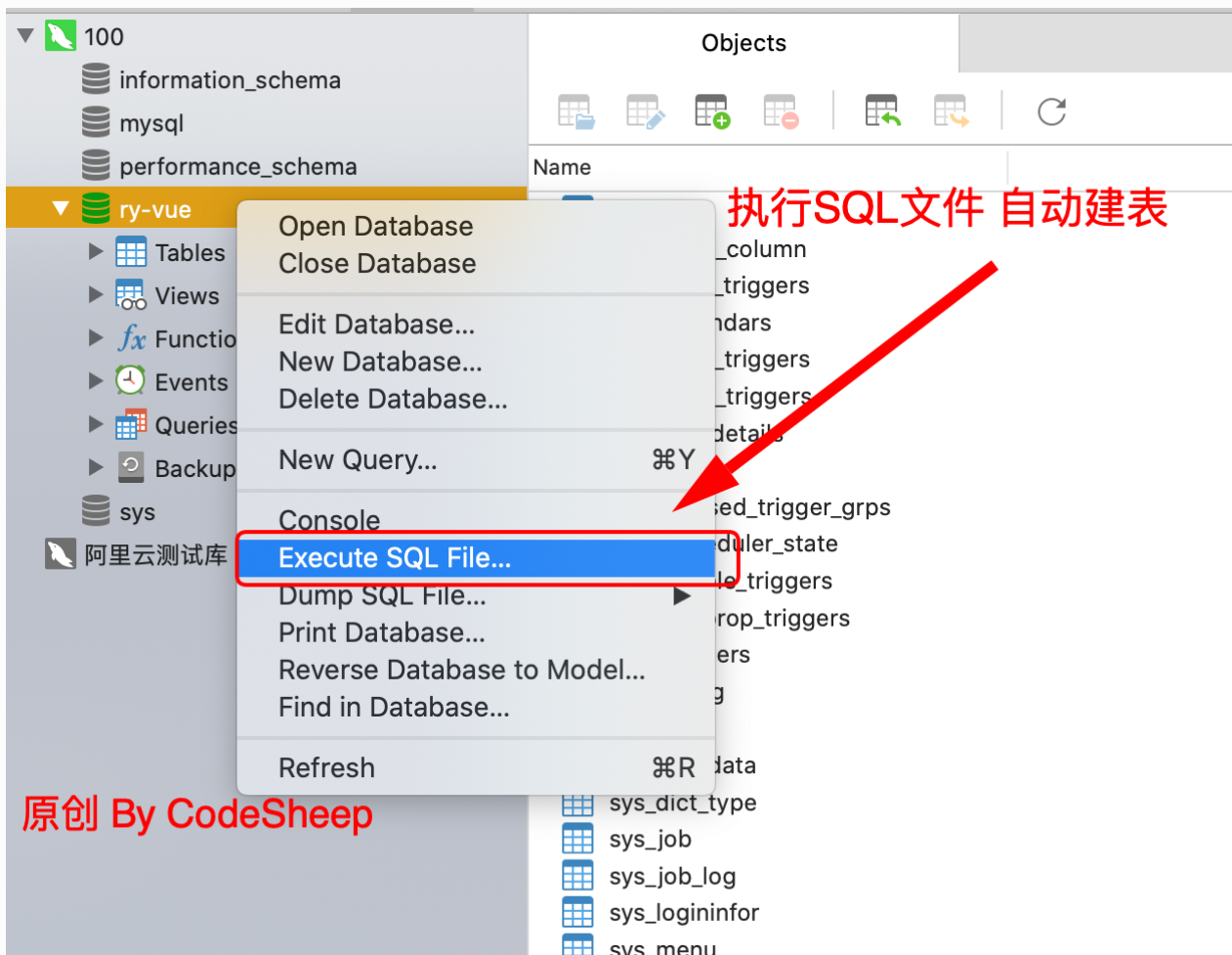
首先我们需要新建一个名为 `ry-vue` 的数据库，出于演示考虑，这里直接通过软件Navicat可视化创建：



2、建表

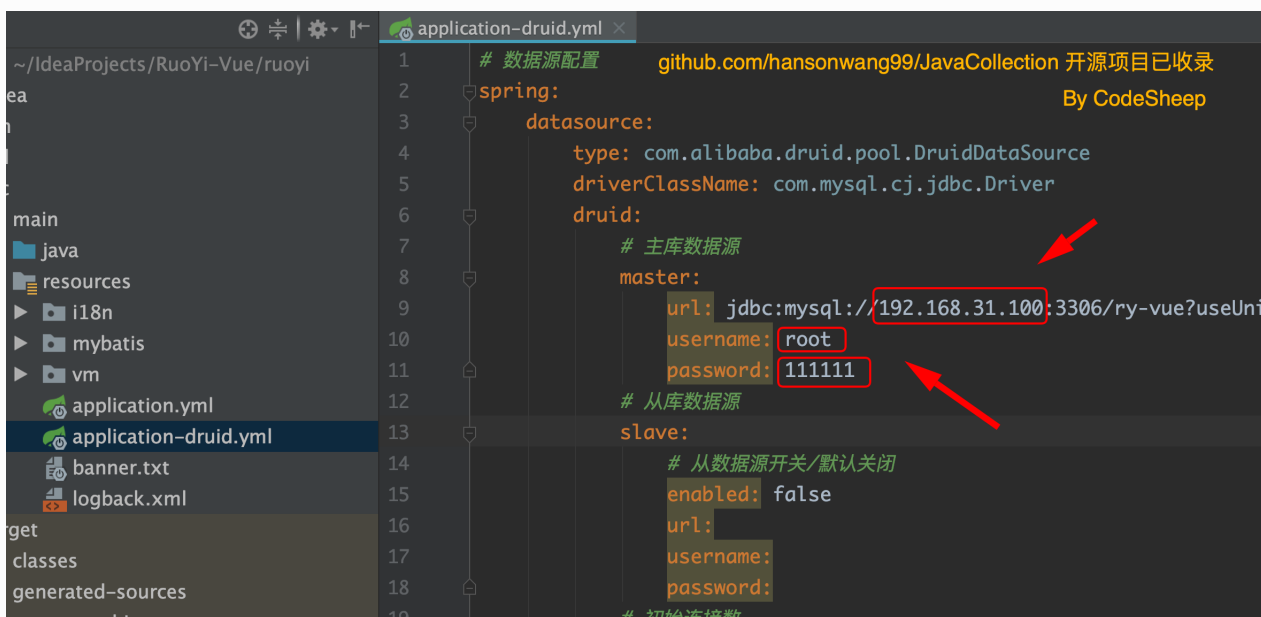
在上述新建的 `ry-vue` 的数据库里导入位于路径 `/Users/codesheep/IdeaProjects/RuoYi-Vue/ruoyi/sql` 下的两个 `.sql` 文件：

- `ry_20200415.sql`
- `quartz.sql`



3、修改yaml中数据库连接配置

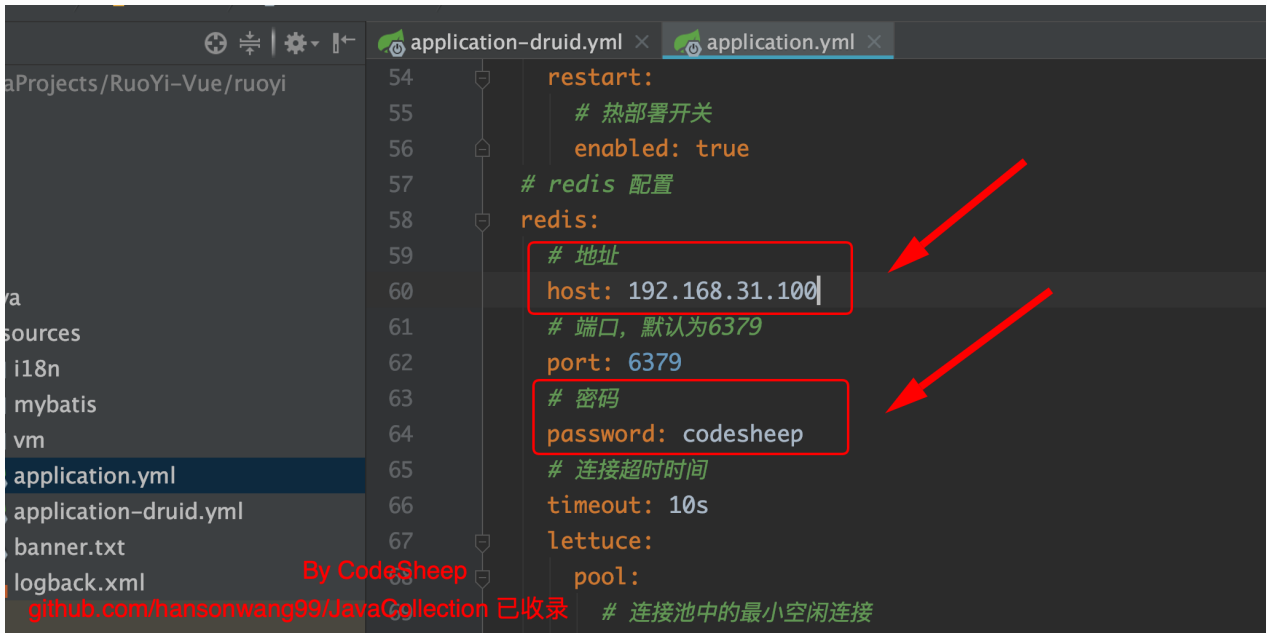
修改路径 `/Users/codesheep/IdeaProjects/RuoYi-Vue/ruoyi/src/main/resources` 下的 `application-druid.yml` 配置文件，修改其中的数据库 `url`、`username`、`password` 等字段为你实际的数据库链接：



修改缓存配置

由于该项目使用了 Redis 缓存，因此和上面的 MySQL 数据库一样需要根据实际情况配置。

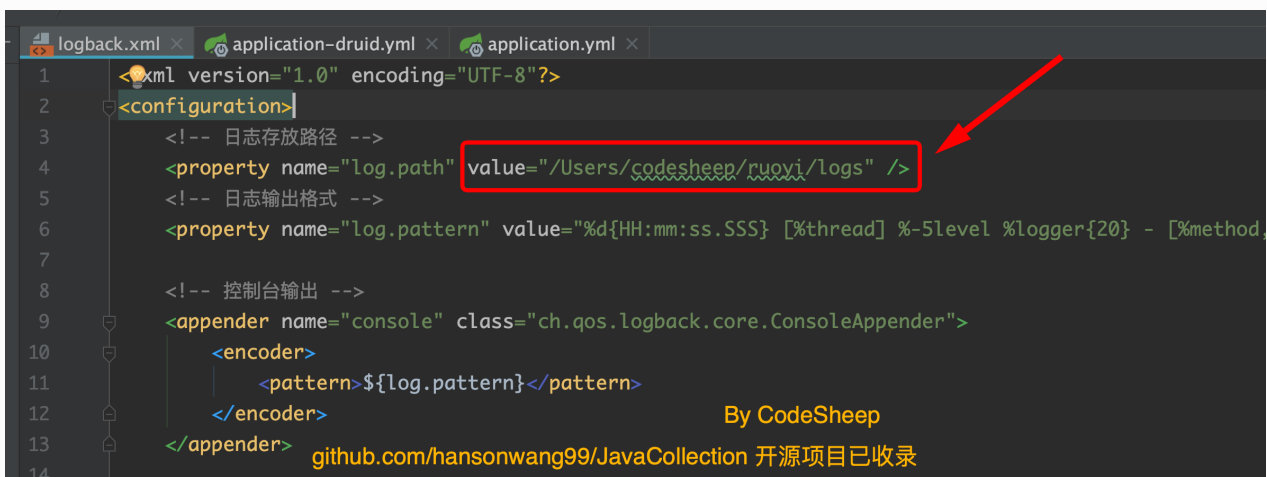
修改路径 `/Users/codesheep/IdeaProjects/RuoYi-Vue/ruoyi/src/main/resources` 下的 `application.yml` 配置文件，修改其中 Redis 缓存的 `host`、`password` 等字段为你实际的 Redis 链接：



修改日志配置

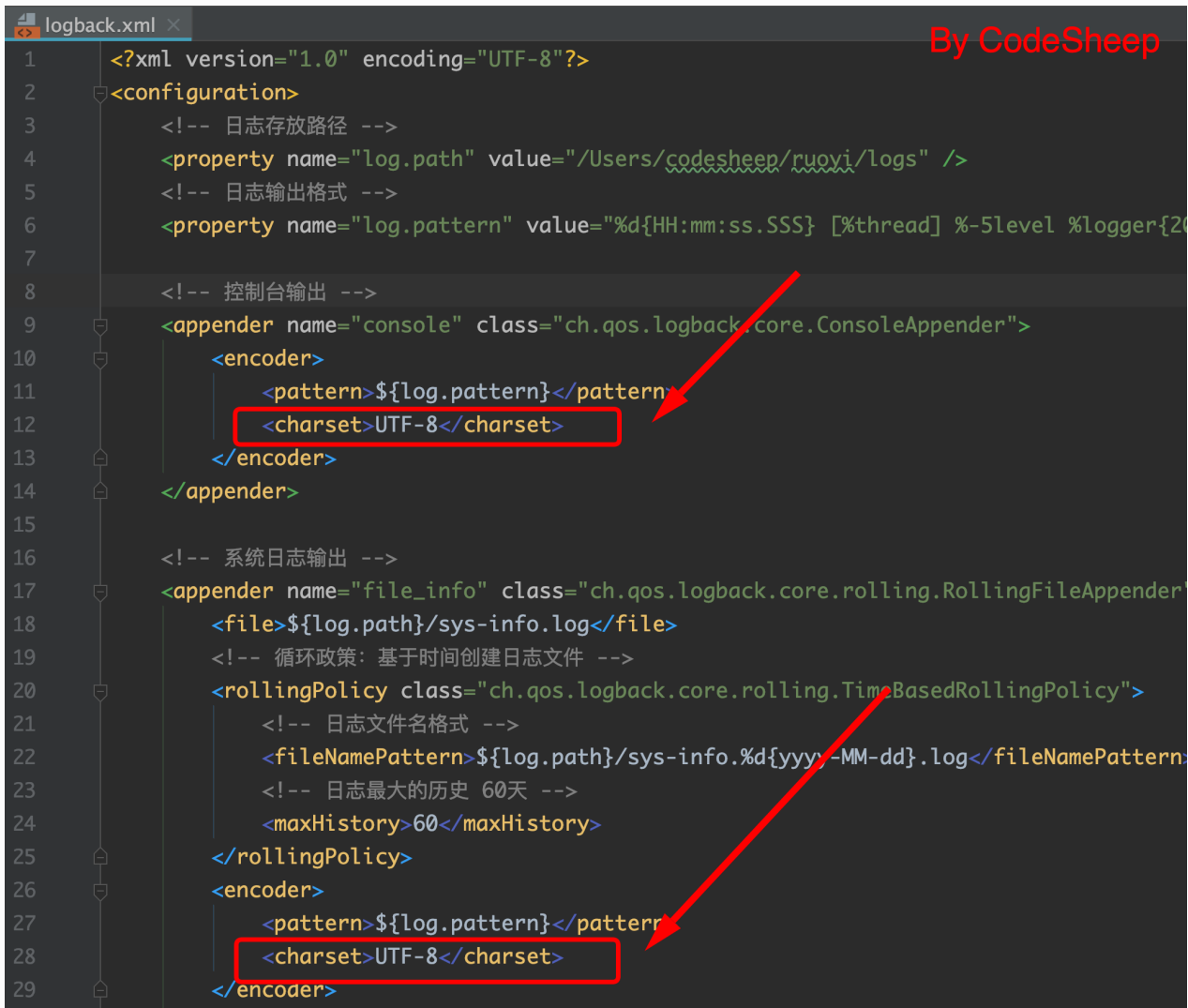
修改日志路径

修改路径 `/Users/codesheep/IdeaProjects/RuoYi-Vue/ruoyi/src/main/resources` 下的 `logback.xml` 配置文件，首先修改其中的日志存放路径为自己定义的路径：



修改字符集

另外则是在每一对 `<encoder></encoder>` 中加上 `UTF-8` 的字符集支持，避免后续服务器上看日志时可能出现乱码的现象：

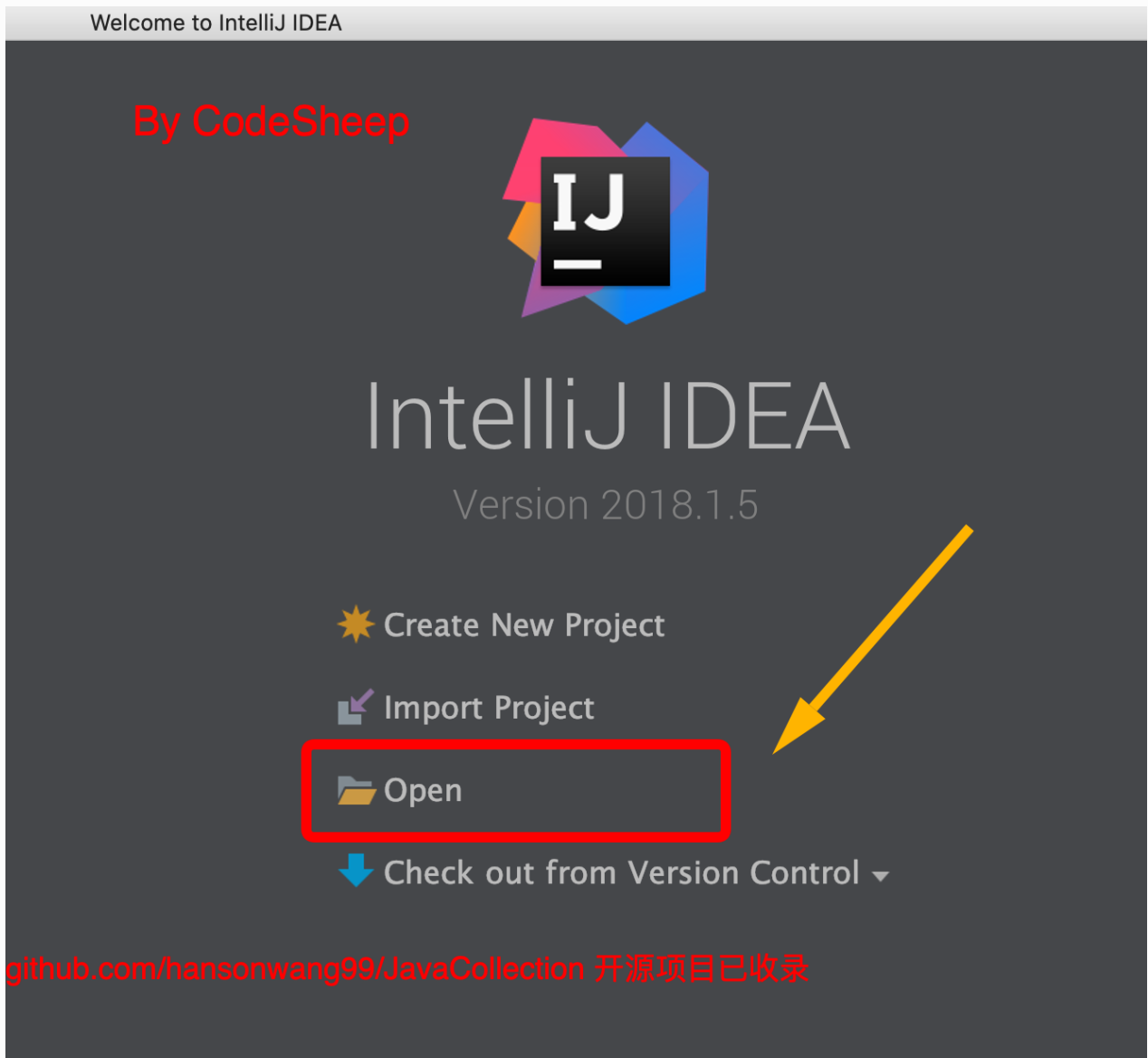


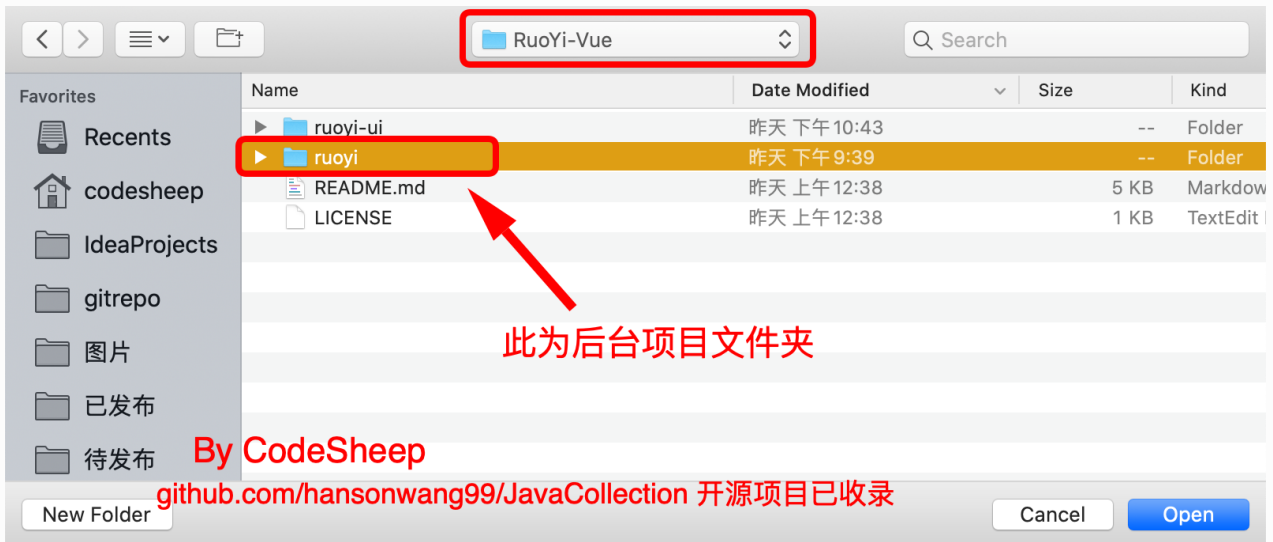
```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <configuration>
3      <!-- 日志存放路径 -->
4      <property name="log.path" value="/Users/codesheep/ruoyi/logs" />
5      <!-- 日志输出格式 -->
6      <property name="log.pattern" value="%d{HH:mm:ss.SSS} [%thread] %-5level %logger{20} %n" />
7
8      <!-- 控制台输出 -->
9      <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
10         <encoder>
11             <pattern>${log.pattern}</pattern>
12             < charset>UTF-8</ charset>
13         </encoder>
14     </appender>
15
16     <!-- 系统日志输出 -->
17     <appender name="file_info" class="ch.qos.logback.core.rolling.RollingFileAppender">
18         <file>${log.path}/sys-info.log</file>
19         <!-- 循环政策：基于时间创建日志文件 -->
20         <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
21             <!-- 日志文件名格式 -->
22             <fileNamePattern>${log.path}/sys-info.%d{yyyy-MM-dd}.log</fileNamePattern>
23             <!-- 日志最大的历史 60天 -->
24             <maxHistory>60</maxHistory>
25         </rollingPolicy>
26         <encoder>
27             <pattern>${log.pattern}</pattern>
28             < charset>UTF-8</ charset>
29         </encoder>
30     </appender>
31 </configuration>
```

By CodeSheep

本地运行

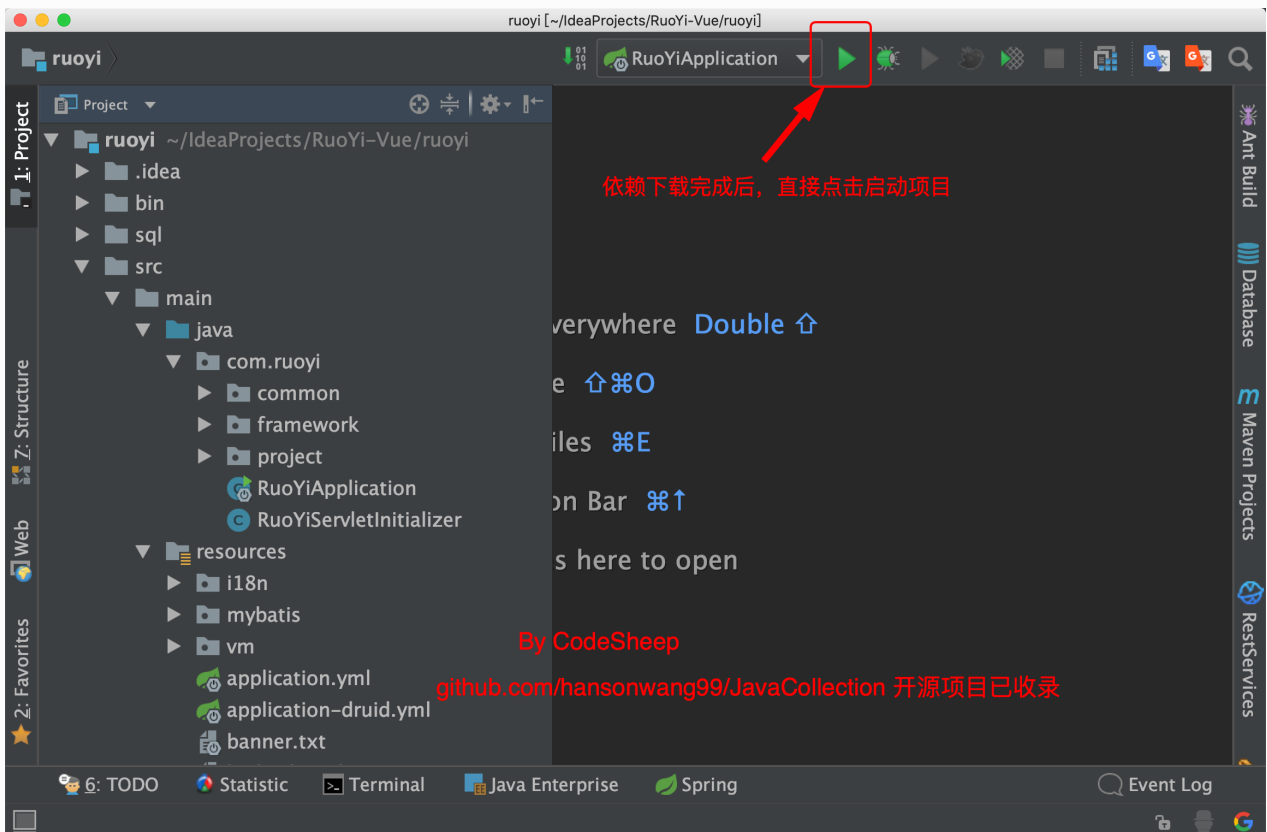
打开 `IntelliJ IDEA`，直接打开本地代码目录 `RuoYi-Vue` 中的 `ruoyi` 文件夹即可：

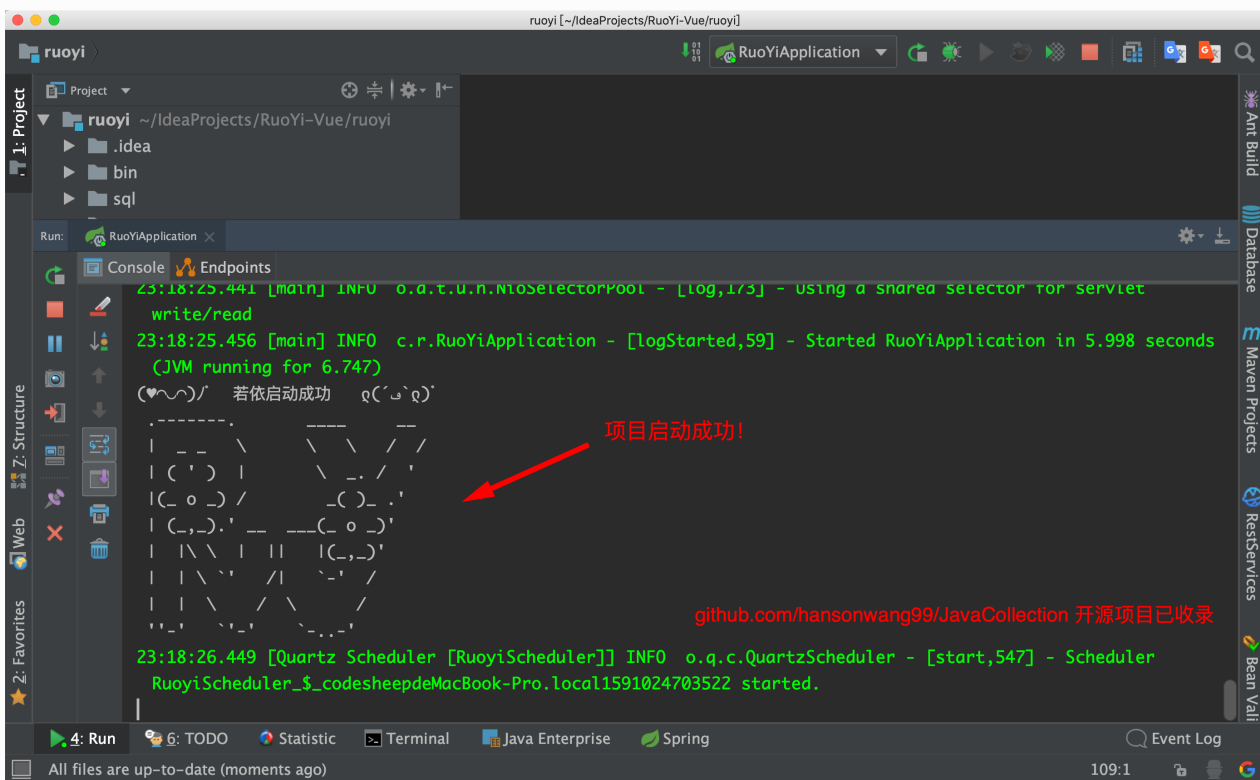




第一次导入，会自动下载项目 `pom.xml` 中各种所需的依赖，所以需要等待一段时间。

导入成功后，直接点击启动按钮，查看启动效果：





项目运行成功，这样后续就可以对其进行调试和代码的研读了。

项目打包

前端项目打包

我们准备将前端项目部署到 `Linux` 节点1 上，所以进入Linux节点1的代码目录：

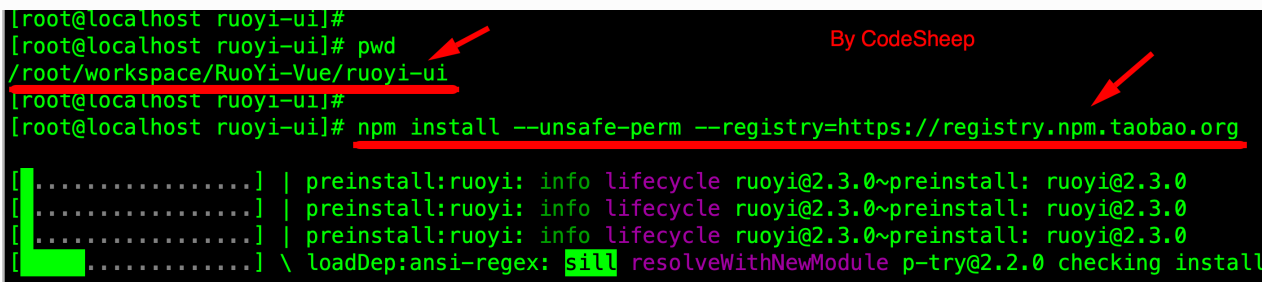
```
/root/workspace/RuoYi-Vue/ruoyi-ui
```

准备进行前端项目的打包操作。

1、安装依赖

进入前端项目子目录 `ruoyi-ui`，运行如下命令安装前端项目所需依赖：

```
npm install --unsafe-perm --registry=https://registry.npm.taobao.org
```

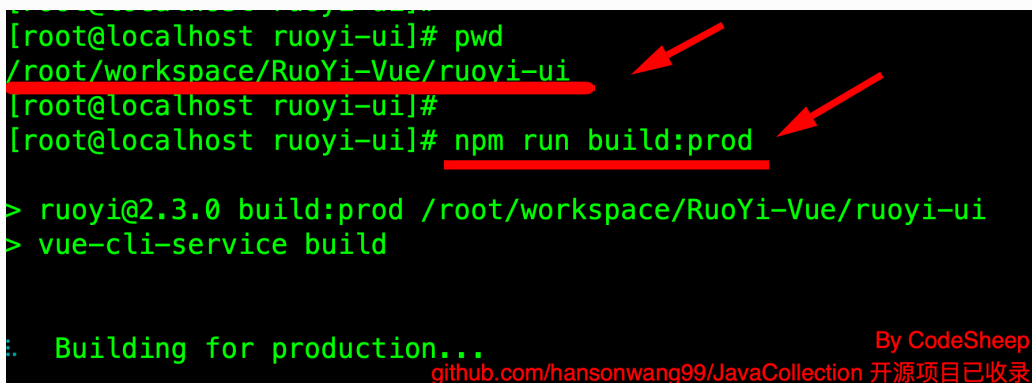


```
[root@localhost ruoyi-ui]#  
[root@localhost ruoyi-ui]# pwd  
/root/workspace/RuoYi-Vue/ruoyi-ui  
[root@localhost ruoyi-ui]#  
[root@localhost ruoyi-ui]# npm install --unsafe-perm --registry=https://registry.npm.taobao.org  
[.....] | preinstall:ruoyi: info lifecycle ruoyi@2.3.0~preinstall: ruoyi@2.3.0  
[.....] | preinstall:ruoyi: info lifecycle ruoyi@2.3.0~preinstall: ruoyi@2.3.0  
[.....] | preinstall:ruoyi: info lifecycle ruoyi@2.3.0~preinstall: ruoyi@2.3.0  
[.....] \ loadDep:ansi-regex: sill resolveWithNewModule p-try@2.2.0 checking install
```

2、构建打包

因为我们要部署，所以这里直接打包正式环境即可：

```
npm run build:prod
```



```
[root@localhost ruoyi-ui]# pwd  
/root/workspace/RuoYi-Vue/ruoyi-ui  
[root@localhost ruoyi-ui]#  
[root@localhost ruoyi-ui]# npm run build:prod  
  
> ruoyi@2.3.0 build:prod /root/workspace/RuoYi-Vue/ruoyi-ui  
> vue-cli-service build  
  
Building for production...  
By CodeSheep  
github.com/hansonwang99/JavaCollection 开源项目已收录
```

构建打包成功之后，会在根目录生成 `dist` 文件夹，里面就是构建打包好的前端项目文件：

```
DONE Build complete. The dist directory is ready to be deployed.
INFO Check out deployment instructions at https://cli.vuejs.org/guide/deployment.html

[root@localhost ruoyi-ui]#
[root@localhost ruoyi-ui]#
[root@localhost ruoyi-ui]# ll
总用量 748
-rw-r--r--. 1 root root 58 6月 1 22:59 babel.config.js
drwxr-xr-x. 2 root root 61 6月 1 22:59 bin
drwxr-xr-x. 2 root root 22 6月 1 22:59 build
drwxr-xr-x. 3 root root 57 6月 2 00:01 dist
drwxr-xr-x. 1147 root root 32768 6月 1 23:52 node_modules
-rw-r--r--. 1 root root 2907 6月 1 22:59 package.json
-rw-r--r--. 1 root root 684594 6月 1 23:51 package-lock.json
drwxr-xr-x. 2 root root 43 6月 1 22:59 public
-rw-r--r--. 1 root root 460 6月 1 22:59 README.md
drwxr-xr-x. 11 root root 203 6月 1 22:59 src
-rw-r--r--. 1 root root 4455 6月 1 22:59 vue.config.js
[root@localhost ruoyi-ui]#
[root@localhost ruoyi-ui]#
[root@localhost ruoyi-ui]#
```

By CodeSheep
github.com/hansonwang99/JavaCollection 开源项目已收录

该 `dist` 目录先留着，后面部署前端项目时，直接拿来使用。

后端项目打包

我们准备将后端项目部署到 `Linux` 节点2和节点3，以节点2为例，进入代码目录：

```
cd /root/workspace/RuoYi-Vue/ruoyi
```

准备进行后端项目的打包操作。

打jar包

`Spring Boot` 由于自带 `Tomcat` 应用服务器，项目默认会打出可执行的 `jar` 包，非常方便。

直接在当前目录下执行 `mvn package` 命令即可构建打包：

```

[root@localhost ruoyi]# github.com/hansonwang99/JavaCollection 开源项目已收录
[root@localhost ruoyi]#
[root@localhost ruoyi]# pwd          路径
/root/workspace/RuoYi-Vue/ruoyi
[root@localhost ruoyi]#
[root@localhost ruoyi]# mvn package 打包命令
[INFO] Scanning for projects...
Downloading from public: http://maven.aliyun.com/nexus/content/grou
2.1.1.RELEASE.pom
Downloaded from public: http://maven.aliyun.com/nexus/content/group
.1.1.RELEASE.pom (12 kB at 12 kB/s)
Downloading from public: http://maven.aliyun.com/nexus/content/grou
1.RELEASE.pom
Downloaded from public: By CodeSheep http://maven.aliyun.com/nexus/content/group
.RELEASE.pom (142 kB at 337 kB/s)
Downloading from public: http://maven.aliyun.com/nexus/content/grou
Downloaded from public: http://maven.aliyun.com/nexus/content/group
Downloaded from public: http://maven.aliyun.com/nexus/content/grou
Downloaded from public: http://maven.aliyun.com/nexus/content/grou

```

该过程会延续一段时间，它会自动下载项目 `pom.xml` 中各种所需的依赖，并最终构建打包。

```

[INFO] Replacing main artifact with repackaged archive
[INFO] -----
[INFO] BUILD SUCCESS ← 编译成功
[INFO] -----
[INFO] Total time: 07:43 min
[INFO] Finished at: 2020-06-02T09:35:59+08:00
[INFO] -----
[root@localhost ruoyi]#
[root@localhost ruoyi]#
[root@localhost ruoyi]# github.com/hansonwang99/JavaCollection 开源项目已收录

```

构建成功如图所示，并且会在当前路径下生成一个 `target` 文件夹：

```

[root@localhost ruoyi]# pwd
/root/workspace/RuoYi-Vue/ruoyi
[root@localhost ruoyi]#
[root@localhost ruoyi]# ll
总用量 16
drwxr-xr-x. 2 root root 64 6月 1 23:00 bin
-rw-r--r--. 1 root root 8408 6月 1 23:00 pom.xml
-rw-r--r--. 1 root root 1697 6月 1 23:00 ry.sh
drwxr-xr-x. 2 root root 47 6月 1 23:00 sql
drwxr-xr-x. 3 root root 18 6月 1 23:00 src
drwxr-xr-x. 6 root root 131 6月 2 09:35 target
[root@localhost ruoyi]#
[root@localhost ruoyi]#

```

进入 `target` 文件夹，构建并打包出来可执行 `jar` 包即位于其中，大小约六十几兆：

```

[root@localhost ruoyi]# cd target/
[root@localhost target]# 
[root@localhost target]# 
[root@localhost target]# 
[root@localhost target]# ll
总用量 67496
drwxr-xr-x. 6 root root      143 6月  2 09:33 classes
drwxr-xr-x. 3 root root       25 6月  2 09:33 generated-sources
drwxr-xr-x. 2 root root       28 6月  2 09:35 maven-archiver
drwxr-xr-x. 3 root root       35 6月  2 09:33 maven-status
-rw-r--r--. 1 root root 68690386 6月  2 09:35 ruoyi.jar
-rw-r--r--. 1 root root 418490 6月  2 09:35 ruoyi.jar.original
[root@localhost target]# 

```

By CodeSheep

打war包

上面说了，`Spring Boot` 由于默认内嵌 `Tomcat` 容器，所以打出来的 `jar` 包即可直接运行，但是我们有时候希望将其部署于外置的 `Tomcat` 服务器中，这时候就要打 `war` 包了。

打 `war` 包之前需要做几个代码上的配置：

1、修改 `pom.xml` 中的打包方式

修改 `/root/workspace/RuoYi-Vue/ruoyi/pom.xml` 文件，将：

```
<packaging>jar</packaging>
```

修改为：

```
<packaging>war</packaging>
```

2、剔除内嵌的Tomcat

我们只需要将 `Spring Boot` 内置的 `Tomcat` 在发布时去除即可。

修改 `/root/workspace/RuoYi-Vue/ruoyi/pom.xml` 文件，在原先的配置：

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

```

下面加入如下配置即可：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
```

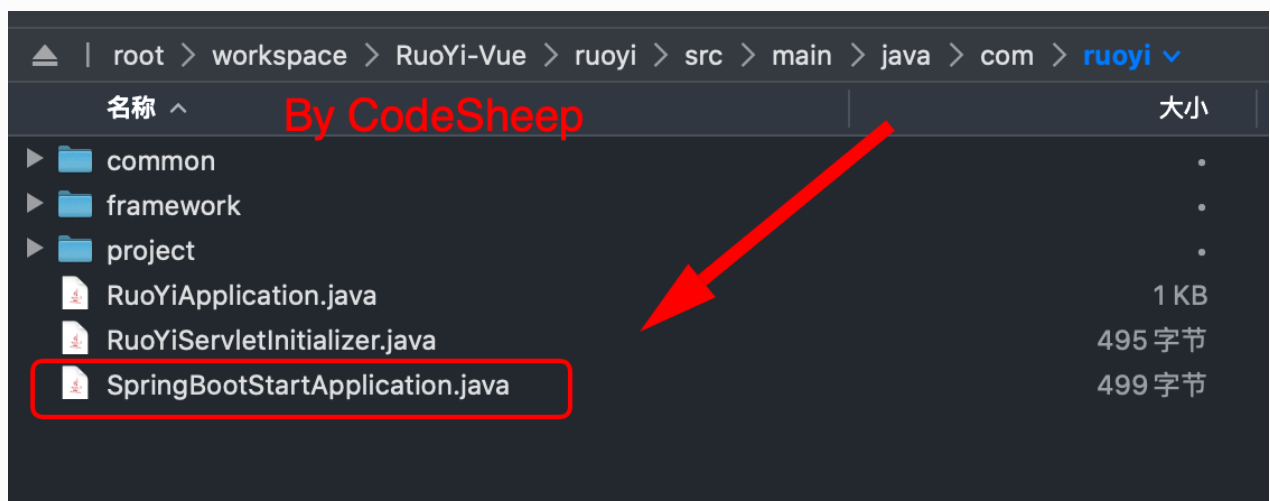
`provided` 表示该包只在编译和测试中使用，在发布时去除。

3、修改项目启动类

在 `Spring Boot` 中我们平常都用 `main` 方法启动的方式，都有一个 `SpringBootApplication` 的启动类，比如本项目的启动主类文件：

```
/Users/codesheep/IdeaProjects/RuoYi-
Vue/ruoyi/src/main/java/com/ruoyi/RuoYiApplication.java
```

而我们现在需要类似于 `web.xml` 的配置方式来启动 `Spring` 应用，为此，我们在 `Application` 类的同级目录下再添加一个 `SpringBootTestApplication` 类：



其代码如下：

```
public class SpringBootTestApplication extends
SpringBootTestServletInitializer {

    @Override
    protected SpringApplicationBuilder
configure(SpringApplicationBuilder builder) {
    // 注意这里要指向原先用main方法执行的 RuoYiApplication启动类
    return builder.sources(RuoYiApplication.class);
    }
}
```

4、执行打包操作

切换到后台项目目录下：

```
/root/workspace/RuoYi-Vue/ruoyi
```

首先执行：

```
mvn clean
```

清理之前的打包结果：

```
[root@localhost ruoyi]# github.com/hansonwang99/JavaCollection 开源项目已收录
[root@localhost ruoyi]# pwd                                     By CodeSheep
/root/workspace/RuoYi-Vue/ruoyi  执行目录
[root@localhost ruoyi]#
[root@localhost ruoyi]#
[root@localhost ruoyi]# mvn clean  执行清理动作
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.ruoyi:ruoyi >-----
[INFO] Building ruoyi 2.3.0
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:3.1.0:clean (default-clean) @ ruoyi ---
[INFO]
[INFO] BUILD SUCCESS ← 清理成功!
[INFO]
[INFO] Total time: 0.370 s
```

然后再次执行 `mvn package` 命令即可构建打war包了，打包结果仍然位于生成的 `target` 目录下：

```
[INFO] Packaging webapp
[INFO] Assembling webapp [ruoyi] in [/root/workspace/RuoYi-Vue/ruoyi/target/ruoyi]
[INFO] Processing war project
[INFO] Webapp assembled in [276 msecs]
[INFO] Building war: /root/workspace/RuoYi-Vue/ruoyi/target/ruoyi.war  打包生成war包
[INFO]
[INFO] --- spring-boot-maven-plugin:2.1.1.RELEASE:repackage (repackage) @ ruoyi ---
[INFO] Replacing main artifact with repackaged archive
[INFO]
[INFO] BUILD SUCCESS ← 构建打包成功
[INFO]
[INFO] Total time: 57.298 s
[INFO] Finished at: 2020-06-02T12:38:56+08:00
[INFO]
[root@localhost ruoyi]# github.com/hansonwang99/JavaCollection 开源项目已收录
[root@localhost ruoyi]# By CodeSheep
```

前端项目部署

上文刚刚阐述过前端项目的打包过程，是在 `Linux` 节点1上进行的，打包好的前端项目成品位于 `dist` 目录，具体路径为：

```
/root/workspace/RuoYi-Vue/ruoyi-ui/dist
```

前端项目的部署非常简单，我们只需要将 `dist` 目录发布到你提前安装部署好的 `Web` 服务器静态目录即可。

我们这里选用的是 `Nginx` 这款高性能Web服务器，接下来我们简单配置下 `Nginx`，并重启生效。

关于Nginx等基础设施和软件的安装部署过程，之前已经专门写过PDF了，可参考前文：[熬10天夜，肝出了这个PDF版「软件安装手册」](#)

我的 `Nginx` 安装于如下目录：

```
/usr/local/nginx
```

其配置文件位于目录：

```
/usr/local/nginx/conf/nginx.conf
```

简要配置 `nginx.conf` 文件如下所示：

```
user root;
worker_processes 1;

#error_log logs/error.log;
#error_log logs/error.log notice;
#error_log logs/error.log info;

#pid logs/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include mime.types;
```



```

default_type  application/octet-stream;

#log_format  main  '$remote_addr - $remote_user [$time_local]
"$request" '
#
#          '$status $body_bytes_sent "$http_referer" '
#          '"$http_user_agent" "$http_x_forwarded_for"';

#access_log  logs/access.log  main;

sendfile      on;
#tcp_nopush   on;

#keepalive_timeout  0;
keepalive_timeout  65;

#gzip  on;

server {
    listen      80;
    server_name localhost;

    #charset koi8-r;

    #access_log  logs/host.access.log  main;

    location / {
        root    /root/workspace/RuoYi-Vue/ruoyi-ui/dist;
        index  index.html index.htm;
    }

    #error_page  404              /404.html;

    # redirect server error pages to the static page /50x.html
    #
    error_page   500 502 503 504  /50x.html;
    location = /50x.html {
        root    html;
    }

    # proxy the PHP scripts to Apache listening on 127.0.0.1:80
    #
    #location ~ /\.php$ {
    #    proxy_pass    http://127.0.0.1;
    #}

    # pass the PHP scripts to FastCGI server listening on
    127.0.0.1:9000
    #
    #location ~ /\.php$ {
    #    root            html;
    #    fastcgi_pass    127.0.0.1:9000;
    #    fastcgi_index   index.php;

```

```

        #    fastcgi_param  SCRIPT_FILENAME
/scripts$fastcgi_script_name;
        #    include        fastcgi_params;
        #}

        # deny access to .htaccess files, if Apache's document root
        # concurs with nginx's one
        #
        #location ~ /\.ht {
        #    deny  all;
        #}
    }

    # another virtual host using mix of IP-, name-, and port-based
    configuration
    #
    #server {
    #    listen      8000;
    #    listen      somename:8080;
    #    server_name  somename  alias  another.alias;

    #    location / {
    #        root    html;
    #        index   index.html index.htm;
    #    }
    #}

    # HTTPS server
    #
    #server {
    #    listen      443 ssl;
    #    server_name localhost;

    #    ssl_certificate      cert.pem;
    #    ssl_certificate_key  cert.key;

    #    ssl_session_cache    shared:SSL:1m;
    #    ssl_session_timeout  5m;

    #    ssl_ciphers  HIGH:!aNULL:!MD5;
    #    ssl_prefer_server_ciphers  on;

    #    location / {
    #        root    html;
    #        index   index.html index.htm;
    #    }
    #}
}

```

此处对原始 `nginx.conf` 文件只做了两处改动：

1、配置 `user root`

2、配置 `location /`

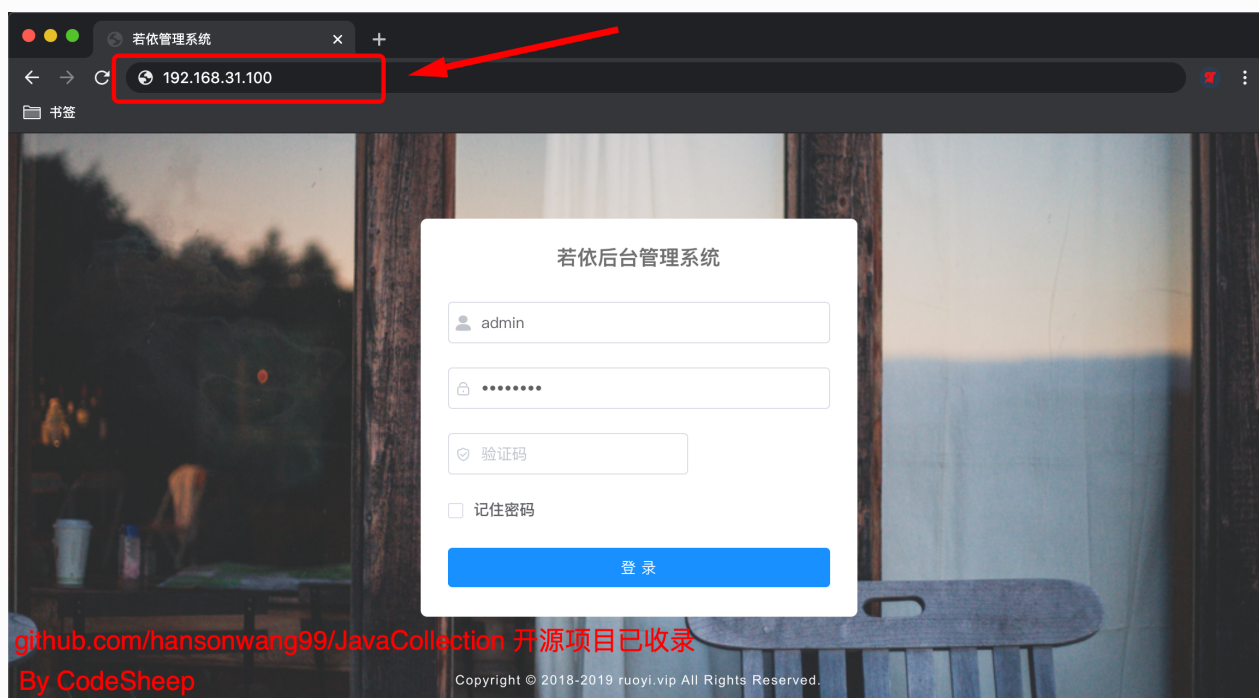
```
location / {  
    root    /root/workspace/RuoYi-Vue/ruoyi-ui/dist;  
    index   index.html index.htm;  
}
```

目的很简单，一眼就能看明白。

接下来执行如下命令重新加载 `Nginx` 即可生效：

```
/usr/local/nginx/sbin/nginx -s reload
```

然后打开浏览器，输入节点1的IP： `192.168.31.100` 即可访问网站前端页面：



至此前端部署结束，接下来进行后端项目的部署。

后端项目部署

JAR包部署方式

`jar` 包部署方式非常简单，几乎只需要一行命令即可搞定，因为 `Spring Boot` 内嵌 `Tomcat`，开箱即用的感觉真的很爽。

上文阐述过后端项目的打 `jar` 包的过程，是在 `Linux` 节点2上进行的，打包好的后端项目 `jar` 包成品位于 `target` 目录，具体路径为：

```
/root/workspace/RuoYi-Vue/ruoyi/target/ruoyi.jar
```

我们只需要进入 `target` 目录中执行如下命令即可在后台部署项目：

```
nohup java -jar ruoyi.jar &
```

```
[root@localhost target]#  
[root@localhost target]# pwd  
/root/workspace/RuoYi-Vue/ruoyi/target 路径  
[root@localhost target]#  
[root@localhost target]# ll  
总用量 67496  
drwxr-xr-x. 6 root root    143 6月  2 11:50 classes  
drwxr-xr-x. 3 root root     25 6月  2 11:50 generated-sources  
drwxr-xr-x. 2 root root     28 6月  2 11:50 maven-archiver  
drwxr-xr-x. 3 root root     35 6月  2 11:50 maven-status  
-rw-r--r--. 1 root root 68690386 6月  2 11:50 ruoyi.jar  
-rw-r--r--. 1 root root  418490 6月  2 11:50 ruoyi.jar.original  
[root@localhost target]#  
[root@localhost target]# nohup java -jar ruoyi.jar &  
[1] 50135  
[root@localhost target]# nohup: 忽略输入并把输出追加到"nohup.out"  
  
[root@localhost target]# By CodeSheep  
[root@localhost target]#
```

WAR包部署方式

war包部署方式就需要额外安装Tomcat应用服务器，我这里的Tomcat安装于如下目录：

```
/usr/local/tomcat/apache-tomcat-8.5.55
```

关于Tocat等基础设施和软件的安装部署过程，之前已经专门写过PDF了，可参考前文：[熬10天夜，肝出了这个PDF版「软件安装手册」](#)

`war` 包的部署方式也非常简单，我们只需要将上文后端项目打包好的 `war` 包直接放入 `Tomcat` 的 `webapps` 目录即可。

上文阐述过后端项目的打 `war` 包的过程，是在 `Linux` **节点2**上进行的，打包好的后端项目 `war` 包成品位于 `target` 目录，具体路径为：

```
/root/workspace/RuoYi-Vue/ruoyi/target/ruoyi.war
```

这里我们直接使用 `cp` 命令拷贝一份到 `Tomcat` 的 `webapps` 目录里即可：

```
cp /root/workspace/RuoYi-Vue/ruoyi/target/ruoyi.war  
/usr/local/tomcat/apache-tomcat-8.5.55/webapps/
```

这时候后端理论上就部署完毕，可以访问了，只不过现在还需要加**项目名**进行访问，就像这样：

```
http://192.168.31.101:8080/ruoyi/
```

很明显这样有点不方便。

所以我们接下来设置去掉项目名，直接根目录访问。

我们只需要修改一下 `Tomcat` 的配置文件 `server.xml`：

```
/usr/local/tomcat/apache-tomcat-8.5.55/conf/server.xml
```

在其 `<Host>` 标签下面加入如下内容即可：

```
<Context path="/" docBase="/usr/local/tomcat/apache-tomcat-  
8.5.55/webapps/ruoyi" reloadable="false"></Context>
```

```

148     <Realm className="org.apache.catalina.realm.UserDatabaseRealm"
149           resourceName="UserDatabase"/>
150 </Realm>
151
152 <Host name="localhost" appBase="webapps"
153       unpackWARs="true" autoDeploy="true">
154
155     <Context path="/" docBase="/usr/local/tomcat/apache-tomcat-8.5.55/webapps/ruoyi" reloadable="false"></Context>
156     <!-- SingleSignOn valve, share authentication between web applications
157          Documentation at: /docs/config/valve.html -->
158     <!--
159     <Valve className="org.apache.catalina.authenticator.SingleSignOn" />
160     -->
161
162     <!-- Access log processes all example.
163          Documentation at: /docs/config/valve.html
164          Note: The pattern used is equivalent to using pattern="common" -->
165     <Valve className="org.apache.catalina.valves.AccessLogValve" directory="logs"
166           prefix="localhost_access_log" suffix=".txt"
167           pattern="%h %l %u %t &quot;%r&quot; %s %b" />
168
169     </Host>
170 </Engine>

```

By CodeSheep
github.com/hansonwang99/JavaCollection 开源项目已收录

最后重启 Tomcat 即可生效：

```

service tomcat stop
service tomcat start

```

配置NGINX代理和转发

接下来我们继续配置节点1上的 Nginx，在其配置文件中加上如下配置：

```

location /prod-api/{
    proxy_set_header Host $http_host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header REMOTE-HOST $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_pass http://192.168.31.101:8080/;
}

```

可以看出来，我们的目的是想将所有发往节点1上的 /prod-api/ 请求，全部转发到节点2上部署的后端项目里。

至于为什么这里配基础转发路径 /prod-api/，这个需要看一下前端项目根目录里的隐藏文件 .env.production 就明白了：

```

[root@localhost ruoyi-ui]# pwd
/root/workspace/RuoYi-Vue/ruoyi-ui 路径
[root@localhost ruoyi-ui]#
[root@localhost ruoyi-ui]# ls -al
总用量 784
drwxr-xr-x.  8 root root  4096 6月  2 00:01 .
drwxr-xr-x.  5 root root   79 6月  1 22:59 ..
-rw-r--r--.  1 root root   58 6月  1 22:59 babel.config.js
drwxr-xr-x.  2 root root   61 6月  1 22:59 bin
drwxr-xr-x.  2 root root   22 6月  1 22:59 build
drwxr-xr-x.  3 root root   57 6月  2 00:01 dist
-rw-r--r--.  1 root root  536 6月  1 22:59 .editorconfig
-rw-r--r--.  1 root root  172 6月  1 22:59 .env.development
-rw-r--r--.  1 root root  111 6月  1 22:59 .env.production
-rw-r--r--.  1 root root  134 6月  1 22:59 .env.staging
-rw-r--r--.  1 root root  307 6月  1 22:59 .eslintignore
-rw-r--r--.  1 root root  5342 6月  1 22:59 .eslintrc.js
-rw-r--r--.  1 root root   279 6月  1 22:59 .gitignore
drwxr-xr-x. 1147 root root 32768 6月  1 23:52 node_modules
-rw-r--r--.  1 root root  2907 6月  1 22:59 package.json
-rw-r--r--.  1 root root 684594 6月  1 23:51 package-lock.json
drwxr-xr-x.  2 root root   43 6月  1 22:59 public
-rw-r--r--.  1 root root  460 6月  1 22:59 README.md
drwxr-xr-x. 11 root root   203 6月  1 22:59 src
-rw-r--r--.  1 root root  4455 6月  1 22:59 vue.config.js
[root@localhost ruoyi-ui]#

```

By CodeSheep

其内容如下：

```

# 生产环境配置
ENV = 'production'

# 生产环境
VUE_APP_BASE_API = '/prod-api'

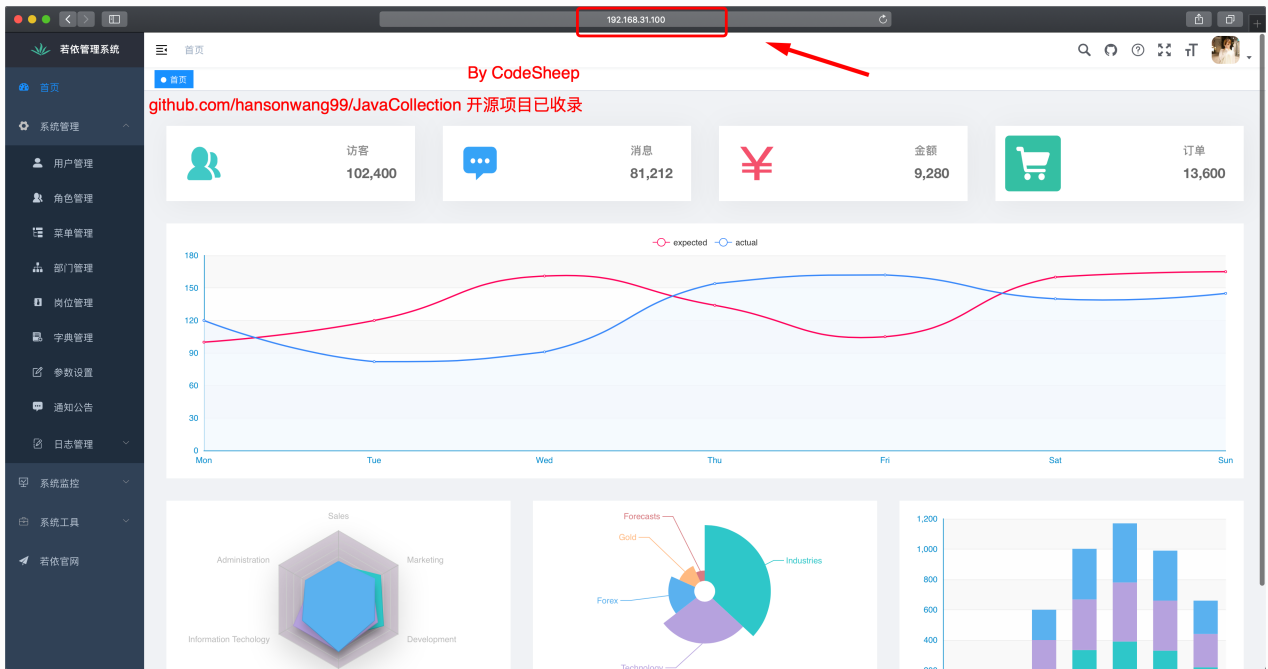
```

这就是需要设置的基础转发路径。

然后接下来执行如下命令重新加载 `Nginx` 即可生效：

```
/usr/local/nginx/sbin/nginx -s reload
```

然后打开浏览器，访问节点1的IP： `192.168.31.100`，即可正常登录，进入项目的后台系统：



至此所有的部署全部结束！

后台项目多实例部署

对于上面的部署，我们对后台项目只部署了一个运行实例，位于**节点2**上。

但实际情况我们处于提升性能或稳定性等考虑，我们希望对后台项目在多个节点上**部署多实例**，所以接下来简要演示一下。

正好我们还准备了一个**节点3**机器，重复上文的后端项目部署过程，在**节点3**上也将后端项目给同样地运行起来。

然后我们继续配置**节点1**上的 `Nginx`，在其配置文件中做如下配置：

```
upstream ruoyi{
    server 192.168.31.101:8080 weight=5;
    server 192.168.31.102:8080 weight=3;
}

server {

    # ... 省略 ...

    location /prod-api/{
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header REMOTE-HOST $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_pass http://ruoyi/;
    }

    # ... 省略 ...
}
```

首先我们配置了一个名为 `ruoyi` 的服务器集群地址，其包含两个实例（分别对应**节点2**和**节点3**），并且分配了不同的权重 `weight`（理论上来说权重 `weight` 越大，对应 `Tomcat` 上分配到的请求也会越多）。

同时将 `location /prod-api/` 里的代理转发路径指向了 `ruoyi` 多实例集群，这样当我们访问前端时，`Nginx` 会自动将我们的请求转发到两个实例中的任何一个中去处理。

这样即使后端实例down掉一个，还有其他实例可以支撑系统的运行，稳定性和性能均要更高一些。

至此，本文档实验过程完全结束！

后记

由于个人精力和能力有限，难免会有疏漏和不当的地方，还希望多多谅解。

另外，联系作者、提意见，可直接扫码联系，一起交流进步：



另外：本文档在 Github开源项目：github.com/hansonwang99/JavaCollection 中已收录，有详细自学编程学习路线、面试题和面经、编程资料及系列技术文章等，资源持续更新中...